OPEN SOURCE APACHE 2.0



AI Gateway & Registry

One Governed Front Door for Any AI Resource: Model Inference, MCP Servers, A2A Agents, Skills, and REST APIs

Now a full **AI gateway and registry**, not just an MCP gateway: since 1.30.0 the same governed front door fronts model inference, MCP servers, A2A agents, and any REST backend.

MODEL INFERENCE MCP SERVERS A2A AGENTS AGENT SKILLS CUSTOM ENTITIES ENTERPRISE READY

GitHub Repository AWS Workshop

Last updated: October 26, 2026

What we'll cover

01 The Problem & the Platform

02 Gateway: Governance & Traffic Control

03 Registry: Discovery, Agents & Federation

04 Operations & Deployment

05 Roadmap & Momentum

06 Get Started

01

The Problem & the Platform

What AI asset sprawl costs you, and the unified control plane that fixes it.

Why We Built This

57 seconds: the problem, how the catalog grew, and the two halves that solve it.



Video not playing? **Watch the 57-second overview**

The AI Asset Sprawl Problem

Teams ship MCP servers, agents, and skills faster than anyone can track. Four needs surface again and again.



Discover

Find the right server, agent, or skill across a fast-growing catalog.



Trust

Know what you connect to is safe before it goes live.



Connect Securely

One front door, instead of every agent holding every credential.



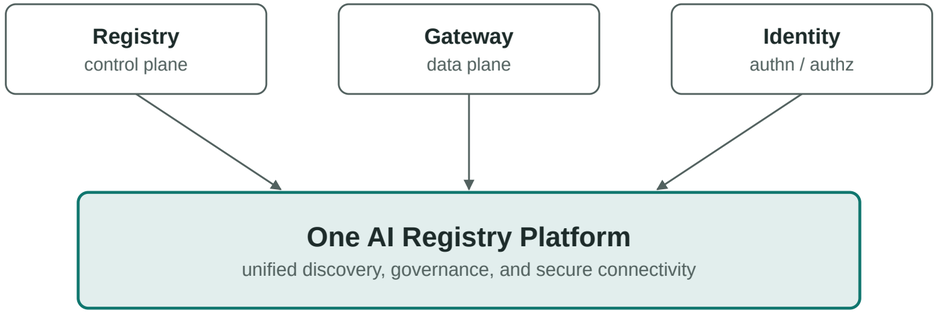
Govern

Control access, versions, and audit across everything.

Point tools solve one need each. The real requirement is a single platform.

What You Really Need: One Platform

A registry, a gateway, and identity for authn and authz, working as one.



Registry

Discover and govern every AI asset: servers, agents, and skills.

Gateway

One secure front door that connects clients and enforces policy.

Identity

OAuth2 across 6 IdPs, with per-user and per-agent access control.

Separately, three problems. Together, one control plane for AI assets.

Five Core Functions

Started as an MCP gateway and registry, now a single solution for any AI asset.

Unified MCP Gateway

Centralized access point for multiple MCP servers with reverse proxy routing, SSL termination, and health monitoring.

MCP Server Registry

Register, discover, and manage MCP servers with unified governance, semantic search, and version routing.

Agent Registry & A2A Hub

Agent registration, discovery, and direct agent-to-agent communication via the A2A Protocol. Supports both A2A and non-A2A agents.

Skills Registry

Register, version, and discover reusable AI skills (SKILL.md) with security scanning, enabling skill sharing across agents and teams.

Custom Entity Registry

Admins define their own entity types with custom schemas, so the registry holds any AI asset: workflows, policies, prompt templates, model cards, and more.

Gateway & Registry

Registry is the control plane, gateway is the data plane -- use them together, or the registry on its own.

Gateway -- the connection plane

A **single endpoint** for every AI resource in your enterprise, not just MCP servers. All the networking plumbing and governance -- reverse-proxy routing, SSL, health, **fine-grained access control**, and **auth to each individual MCP server** -- is handled by the gateway, so clients connect once and the gateway brokers the rest.

Per-user egress auth (3LO + OBO), A2A reverse-proxy routing, and application-level rate limiting -- all shipped in 1.27.0. Any-resource routing (inference, MCP, A2A, REST) shipped in 1.30.0.

Registry -- the discovery plane

Your **discoverability control plane** for humans, AI agents, and AI coding assistants:

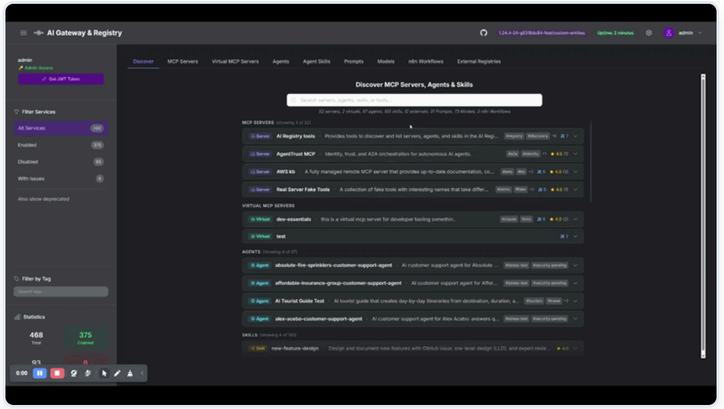
- Add the **registry MCP server** and your coding assistant has the full registry at its fingertips.
- Agents call the **search API** (REST, or via the AI-registry-tools MCP server) to **discover tools and assets at runtime** and use them.
- Register **any type of asset** -- MCP servers, agents, skills, and custom entities.

The default is gateway + registry combined -- but you can run it as a registry only.

From MCP Registry to AI Asset Registry

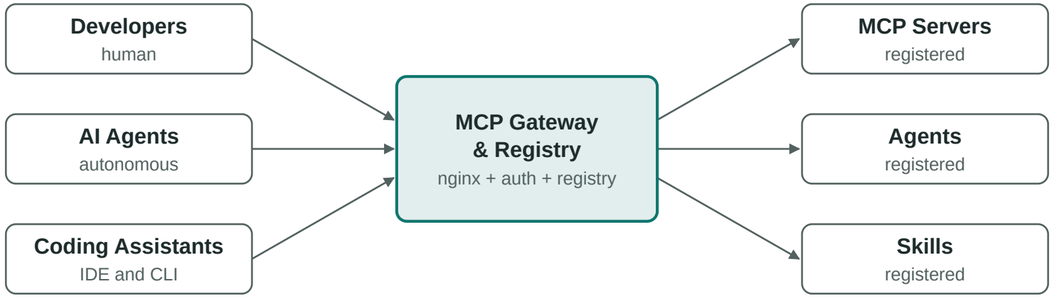
It started with MCP servers, then agents and skills. Now admins define **custom entity types** with their own schema, so the same registry holds *any* AI asset your teams produce.

N8N WORKFLOWS | POLICIES | PROMPT TEMPLATES | MODEL CARDS | DATASETS | EVALUATIONS



Each custom entity type gets its own tab, schema-driven registration, semantic + lexical search, voting, and registry cards, with the same governance and audit trail as everything else.

Platform Architecture



Servers and agents can be hosted **anywhere**:

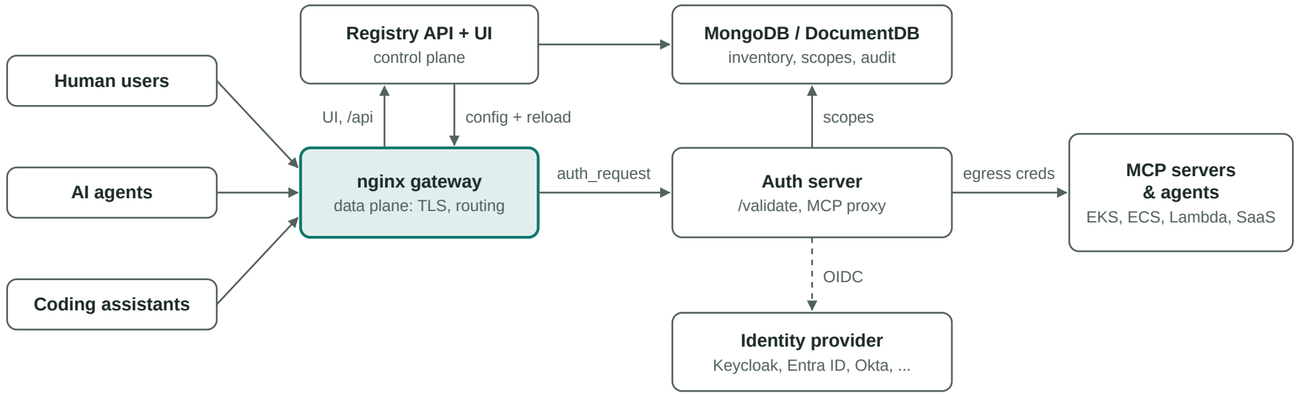
AMAZON BEDROCK AGENTCORE | AMAZON EKS | OTHER AWS COMPUTE | THIRD-PARTY SAAS

Skills are sourced from **GitHub or private Git repositories** (PAT or GitHub App auth, GitHub Enterprise supported).

Single connection point with **dual authentication** for human users and machine-to-machine agent flows.

Inside the Gateway

The registry is the control plane and nginx is the data plane. Every MCP call is checked and proxied by the auth server.



The registry decides what nginx may route to. nginx and the auth server enforce it on every call, and per-user egress tokens stay in the vault instead of on laptops.

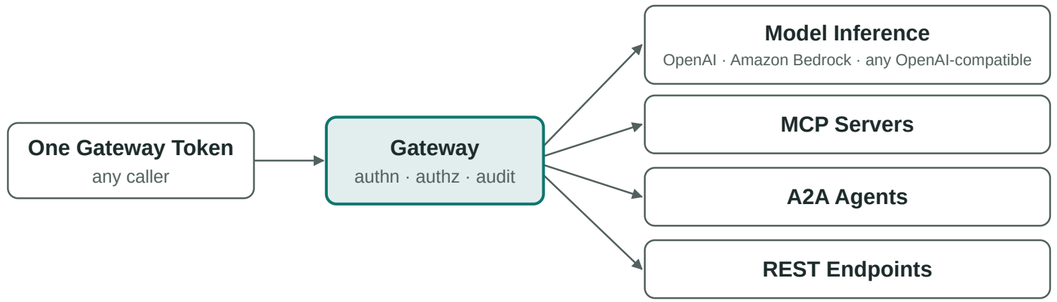
02

Gateway: Governance & Traffic Control

One ingress that authenticates, authorizes, brokers egress identity, and steers AI traffic.

One Gateway for Any Resource

New in 1.30.0: the gateway fronts any HTTP backend, not only MCP servers. One token and one URL shape cover four kinds of traffic.



Keys never leave the registry

The backend URL and its credential stay in the registry, encrypted at rest and injected on egress, so no provider key lands on a laptop. Four credential models: keyless, operator key, caller passthrough, per-user vault.

Streaming, with ceilings

Per-entity response streaming carries SSE and Amazon Bedrock event-stream bodies incrementally, under bounded concurrency, byte, and duration limits.

The same governance

The same scopes, audit trail, and rate limits as the rest of the registry. A legacy `methods: ["all"]` grant authorizes no HTTP verb, so nothing gains access by accident.

Callers reach everything at `/gateway/{type}/{name}`. Off by default, per-entity opt-in, no schema migration.

Enterprise Security

Multi-Provider IAM

Keycloak, Microsoft Entra ID, Okta, Auth0, and AWS Cognito with harmonized management API.

Fine-Grained Access

Scopes define which servers, methods, tools, and agents each user or service account can access.

Security Scanning

Cisco AI Defense integration for MCP servers and A2A agents with YARA pattern matching.

Audit Logging

Comprehensive audit trails with credential masking, TTL retention, and SOC 2/GDPR compliance.

M2M Auth

OAuth2 Client Credentials flow for AI agent identity with service accounts.

Static Token Auth

API key access for CI/CD pipelines and trusted network environments without IdP setup.

Per-User Egress Auth: 3LO & OBO

SHIPPED · 1.27.0

The gateway brokers each user's own identity to third-party backends -- no shared service credentials, no tokens in agent code.

3LO -- Bring Your Own Account

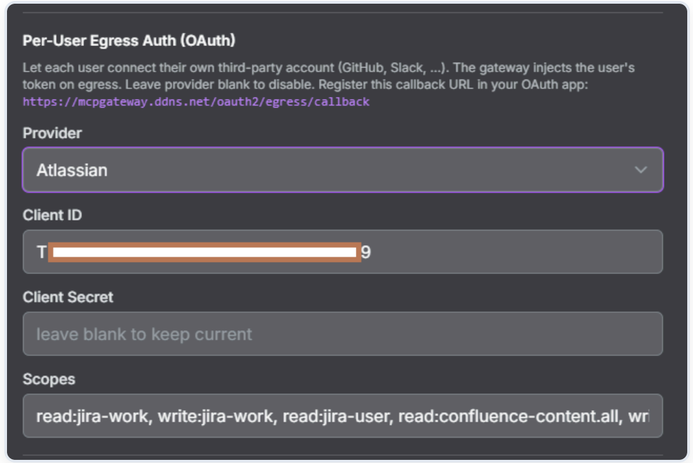
Each user connects their own GitHub, Slack, Atlassian, or Google account once (three-legged OAuth). The gateway vaults the token in OpenBao / AWS Secrets Manager and injects it on egress.

OBO -- On-Behalf-Of Exchange

For same-trust-domain backends, the gateway performs RFC 8693 token exchange so the downstream server sees the real user identity. Target audiences are an audited, explicit allowlist.

Per-User, Auditable, Revocable

Tokens are scoped to the individual, masked in logs, and revoked by disconnecting the account. Direct PAT egress is next.



Per-user egress auth config -- connect a provider, vault the token, inject on egress

OBO: Call Internal Servers as the Real User

SHIPPED · 1.27.0

3LO brokers a third-party account the user owns. OBO covers the other half: internal servers that already trust the same identity provider as the gateway.

Same IdP, No Consent, No Vault

The gateway takes the user's ingress JWT and re-audiences it at its own identity provider (Entra `jwt-bearer`, Keycloak RFC 8693), minting a fresh token per request. Nothing gets stored, and the user sees no consent prompt.

The Backend Sees the Real User

What arrives is an IdP-signed JWT carrying the user's own `sub`, audienced to that MCP server's app. The server can exchange it again for a downstream resource such as Microsoft Graph and call that resource as the user.

Audience and Scope on an Allowlist 1.31.0

Every exchange checks `target_audience` and its scopes against an explicit allowlist, on the registration path and the vend path alike. The audit trail names an identifiable user in place of an opaque `sub`.

Two modes, one injection seam

	3LO vault	OBO exchange
Token	Vaulted per user	Minted per request
IdP	Third party	Same as gateway
Consent	Once per server	None
Storage	OpenBao / Secrets Manager	Stateless
Targets	External SaaS	Internal, same tenant

Both modes share the same egress injection seam and differ only in where the forwarded token comes from. Set it per server with `egress_auth_mode`.

Egress Auth in Action

A user connects their own account, then an agent calls the backend on their behalf -- token vaulted and injected by the gateway.

Video not playing? **Watch the egress auth demo**

A2A Reverse-Proxy Routing

SHIPPED · 1.27.0

Optionally route inter-agent (agent-to-agent) traffic through the gateway too, so agent calls get the same fine-grained access control we already apply to MCP servers.

One Endpoint for Agents

The gateway fronts your A2A agents the same way it fronts MCP servers -- callers hit a single ingress and the gateway brokers the rest. The agent's real backend stays private.

Per-Call FGAC

Each agent is gated per call with its own `invoke_agent` grant. Access decisions are enforced at the gateway, not left to the agent.

Same Governance as MCP

Unified auth, audit, and scoping across MCP servers and A2A agents -- one control plane for every hop of agent traffic.

Inter-agent calls now inherit the gateway's **fine-grained access control, audit, and identity brokering** -- opt-in, backends stay private.

Application-Level Rate Limiting

SHIPPED · 1.27.0

The gateway now does AI traffic management -- identity-, group-, and target-aware request limits enforced at the auth-server hop, complementary to coarse per-IP edge limiting.

Cap the Caller

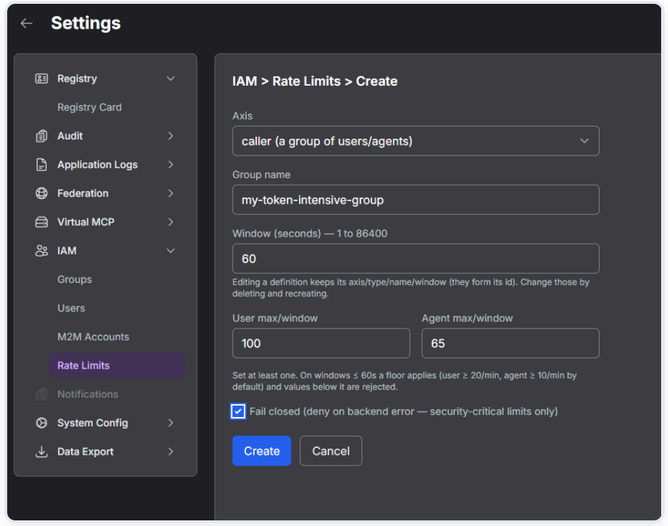
Limit a group of users or agents (M2M) per window. Separate user and agent ceilings, with lockout-safeguard floors so you can never accidentally throttle a group to zero.

Protect the Target

Cap requests to a busy MCP server or A2A agent regardless of who is calling, so one hot backend can't be overwhelmed.

Fail-Open or Fail-Closed

Global fail-open by default so a counter-store blip never becomes a gateway outage; flip security-critical limits to fail-closed per definition. Managed at runtime via API, CLI, or the UI.



Settings → IAM → Rate Limits -- per-group user and agent ceilings

Virtual Servers & Version Routing

Virtual MCP Servers

- ■ Aggregate tools from multiple backends
- ■ Tool aliasing for naming conflicts
- ■ Per-tool scope-based access control
- ■ Session multiplexing (1:N transparent)
- ■ 60s cached aggregation

Version Routing

- ◆ Multiple versions behind single endpoint
- ◆ Test with X-MCP-Server-Version header
- ◆ Instant rollback with one API call
- ◆ Deprecation lifecycle with sunset dates
- ◆ O(1) nginx map-based routing

03

Registry: Discovery, Agents & Federation

Find, trust, and import any AI asset -- for humans, agents, and coding assistants.

Intelligent Discovery

Hybrid Search

- ◆ Vector similarity + keyword matching
- ◆ Unified search across servers, tools, agents, skills
- ◆ Name-boost for exact match relevance
- ◆ Tag-based and metadata filtering

Flexible Embeddings

- ■ Local sentence-transformers
- ■ OpenAI embeddings
- ■ Amazon Bedrock Titan
- ■ 100+ LiteLLM-supported providers

Agents discover tools at runtime via natural language -- **"find an agent that can book flights"**

One-Command OAuth for Coding Assistants

DCR (Dynamic Client Registration) and CIMD (Client ID Metadata Document) enable zero-config authentication. The assistant registers itself, the user clicks Allow, done.

DCR (RFC 7591)

Assistant self-registers with Keycloak via the gateway's published OAuth metadata. No pre-provisioned client IDs needed. Per-user tokens with group-based entitlements.

Works today: Claude Code, Codex CLI, Claude.ai Connectors, Kiro

CIMD (draft-parecki)

For IdPs without DCR (Microsoft Entra ID): client publishes its metadata at a URL the IdP fetches. Same one-command UX, different wire protocol. Gateway acts as CIMD consumer.

Status: Phase 4 of umbrella #988

▶ Claude Code Demo **▶ Codex CLI Demo** **Umbrella Issue #988**

Agent Registry & A2A Protocol

Agent-to-Agent Communication

Direct peer-to-peer agent communication. Registry handles discovery, agents connect directly for low-latency interaction.

Semantic Agent Discovery

Agents find collaborators via natural language queries. Dynamic composition based on capabilities, not hardcoded references.

Agent Skills Registry

Register reusable SKILL.md instruction sets with security scanning, visibility controls, and star ratings.

Agents autonomously discover specialized agents for tasks they cannot handle, enabling **dynamic agent orchestration**.

Supply Chain Security: Cisco AI Defense

EXTERNAL INTEGRATION

Three open-source scanners automatically scan every MCP server, A2A agent, and Agent Skill before it reaches production.

MCP Scanner

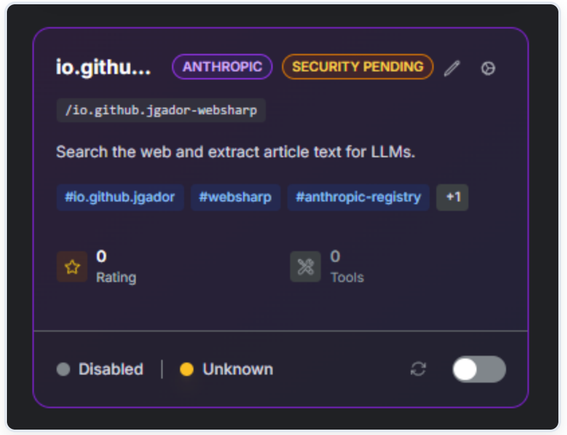
YARA pattern matching and optional LLM analysis for MCP server tool definitions. Detects injection, XSS, and hardcoded secrets.

A2A Scanner

Heuristic and spec validation for Agent-to-Agent protocol agents. Catches protocol violations and malicious behaviors.

Skill Scanner

Static and behavioral analysis of SKILL.md files. Detects prompt injection, privilege escalation, and social engineering.



Unsafe items are auto-disabled with a "security-pending" tag for admin review

Scan on registration, on-demand rescan, and periodic registry-wide audits -- powered by cisco-ai-defense open-source scanners

Agent Trust: Agent Name Service

EXTERNAL INTEGRATION

DNS + PKI for AI Agents -- just as SSL certificates let you trust websites, ANS lets you trust agents.

Structured Identity

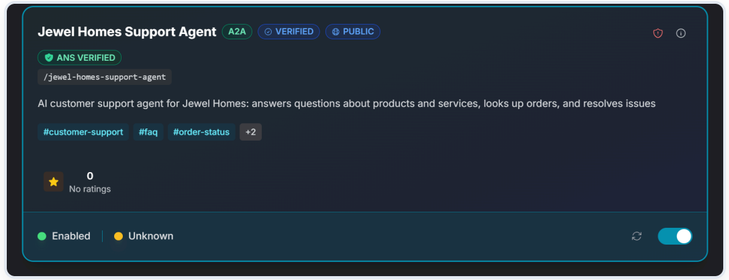
Each agent gets an ANS Name and X.509 certificate issued by a Registration Authority.

Bring Your Own ANS ID

Owners link their ANS Agent ID to registry entries. Registry verifies via GoDaddy ANS API.

Production Resilience

Circuit breaker, retry with backoff, 6-hour background re-verification, and rate limiting.



Verified agents display a trust badge with certificate details

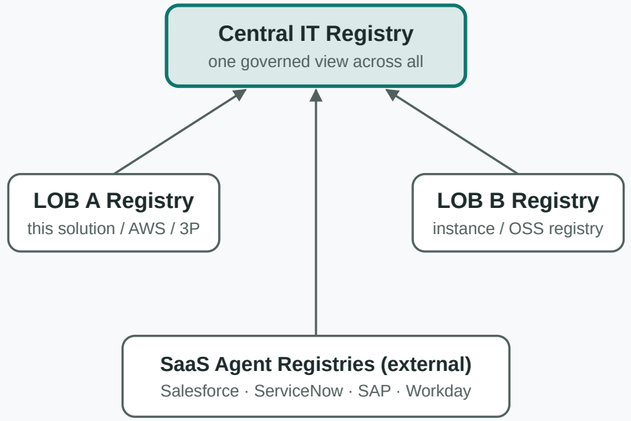
IETF Internet-Draft (draft-narajala-ans-00) -- Powered by GoDaddy ANS, integrated into the open-source AI Registry

Federation: Two Scenarios

CONCEPT

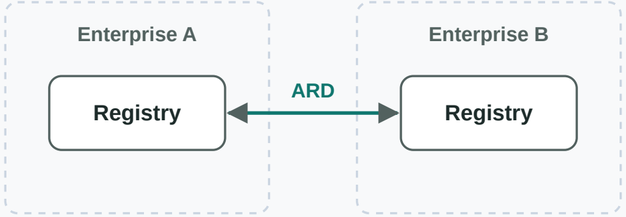
Federation links registries so AI assets are discoverable across boundaries. It appears two ways: **within one enterprise** and **across enterprises**. Any participant can be an instance of this solution or any ARD-compliant registry.

Within a Single Enterprise



Central IT gains one governed view across line-of-business registries and SaaS agent systems of record.

Across Enterprises



any ARD-compliant registry participates

Two organizations discover each other's AI assets over ARD, with no shared infrastructure required.

Participants can be anything ARD-compliant: instances of this registry, AWS Agent Registry, Anthropic, Workday ASOR, or other third-party and open-source registries.

Agentic Resource Discovery (ARD)

Full **ARD v1.0** implementation -- vendor-neutral, standards-track discovery so any ARD-aware client, agent, or registry can find, search, and cross-reference assets.

Publisher

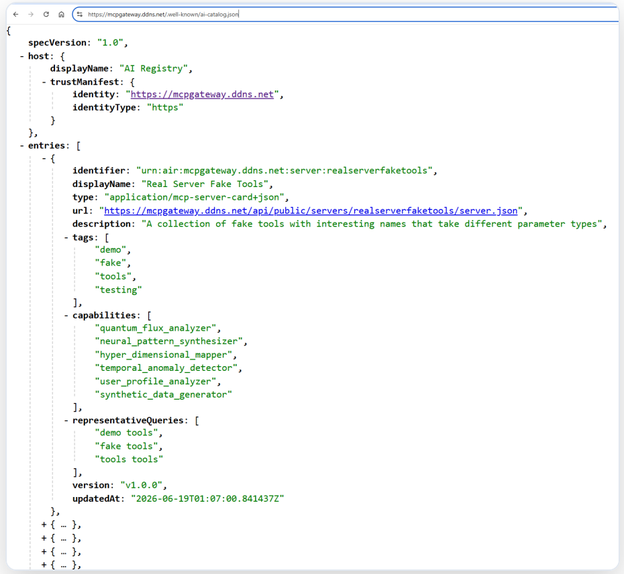
Anonymous, crawlable **/.well-known/ai-catalog.json** of public + enabled assets, each a domain-anchored URN with the correct IANA media type.

Registry

Search & browse: **POST /api/ard/search** and **GET /api/ard/agents** -- JWT-required and access-scoped, wrapping the same semantic engine.

Federation

Ingest any ARD-conformant registry's **ai-catalog.json** into one unified index, with domain-anchored trust and an SSRF-guarded crawler.



Live **/.well-known/ai-catalog.json**

A format adapter over existing registry data -- discover → resolve → connect, with no new storage.

Listed as an ARD reference implementation

External & Peer Registries

Two kinds of federation partner: **peer registries** you operate, and **external registries** you integrate.

Peer Registries

P2P Federation

Bidirectional sync between registry instances, with configurable modes: all, whitelist, or tag filter.

Registry Card

Standardized discovery via **/.well-known/registry-card** advertising capabilities, auth endpoints, and contact info.

Peers are instances of this solution or any ARD-compliant registry.

External Registries

AWS Agent Registry

Federate MCP servers, A2A agents, and skills. Multiple registries across accounts and regions (cross-account via IAM role assumption), selective sync, and cascade cleanup.

Anthropic Registry

Import curated MCP servers from **registry.modelcontextprotocol.io**. Public API, no auth: set `ANTHROPIC_REGISTRY_ENABLED=true`.

Workday ASOR & SaaS

Federate Workday-managed agents (Agent System of Record) via OAuth, mapped to A2A agent cards. Salesforce, ServiceNow, and SAP plug in through the same adapter interface.

Unified governance: federated assets inherit the same scanning, RBAC, and audit as locally registered items. Enable each source with a single env var (for example **AWS_REGISTRY_FEDERATION_ENABLED=true**).

04

Operations & Deployment

Observability, flexible deployment targets, and the developer experience.

Observability & Metrics

Grafana Dashboards

Real-time metrics for server health, tool usage, auth events, and system performance.

OTLP Push Export

Direct export to Datadog, New Relic, Honeycomb, or Grafana Cloud -- no OTEL Collector needed.

OpenTelemetry

Integration with Prometheus, CloudWatch, and other monitoring platforms via OTEL protocol.

Anonymous Telemetry

Privacy-first usage telemetry for adoption tracking. Non-sensitive metadata only. Opt-out with one env var.

Observability Guide: metric reference, queries, and alerts

Flexible Deployment

Amazon EKS (Kubernetes)

Helm chart deployment on Amazon EKS for container orchestration at scale.

Amazon EC2

Traditional deployment on EC2 instances with Docker Compose. Complete setup guide from scratch.

Docker / Podman

Pre-built images on Amazon ECR Public, plus rootless Podman support for macOS and Linux. One-command AI-assisted setup.

Amazon ECS Fargate

Serverless containers with multi-AZ, ALB with HTTPS, auto-scaling, CloudWatch, and NAT Gateway HA. Full Terraform config.

4	6	8,815
Deployment Targets	Identity Providers	Passing Tests (35 releases)

Helm charts work on any Kubernetes cluster: AWS EKS, Azure AKS, GCP GKE, or your own data center.

Developer Experience

MCP Registry CLI

Conversational interface for registry management with real-time token status and cost tracking.

IAM Settings UI

Visual management of users, groups, and M2M service accounts with fine-grained permissions.

AI Coding Assistants

One-command DCR OAuth for Claude Code, Codex, Kiro, and Claude.ai. No tokens to paste, no client IDs to provision.

Management API

RESTful HTTP endpoints with Python client, type-safe interfaces, and comprehensive error handling.

Config Viewer

Admin panel with 11 config groups. Export as ENV, JSON, TFVARS, or YAML for deployment automation.

Custom Metadata

Rich metadata on servers and agents for team ownership, compliance tracking, and cost allocation.

05

Roadmap & Momentum

What just shipped, what is coming next, and the community behind it.

Where We Are Today

As of September 2026. The capabilities that matter, tagged by the release that shipped them. Every one is in a tagged, documented release on GitHub.

1.24.4	**A registry for any AI asset.** Custom entity types with their own schema, hybrid (RRF) semantic search, and native OpenTelemetry metrics.
1.24.5-7	**Deploy it your way.** Docker Compose, Terraform on ECS, CDK, and Helm on EKS, with six identity providers to choose from.
1.25.0	**Agentic Resource Discovery (ARD v1.0).** Publish, search, and ingest asset catalogs across registries, vendor neutral.
1.25.0	**One-command OAuth for coding assistants.** Claude Code, Codex CLI, Kiro, and Claude.ai Connectors attach with no manual token handling.
1.26.0	**Per-user egress credential vault.** Each user connects their own GitHub, Slack, or Atlassian account once; the gateway vaults it and injects it on egress.
1.27.0	**A2A routing, OBO, and rate limiting.** Inter-agent traffic gets per-call access control, backends are called as the real user, and limits are identity and target aware.
1.28-29	**One fail-closed egress guard.** Every outbound fetch runs through the same SSRF guard, credentials are bound to the exact validated destination, and token lifetimes are operator-tunable.
1.30.0	**One gateway for any resource.** Model inference, MCP servers, A2A agents, and REST endpoints behind one token and one URL shape.
1.31.0	**Backend identity and tool-level security.** The registry onboards servers that demand OAuth before they answer, blocks a flagged tool instead of the whole server, and cuts search latency from 641ms to 86ms with graded relevance scores.

One governed front door for every AI resource, with per-user identity end to end.

Security has been our single biggest investment over the last six months, and it never stops

One fail-closed SSRF and egress guard on every outbound fetch. Shared CSRF protection on state-changing routes. Token-free response projection with per-caller redaction on every read path. Hardened OAuth: redirect allowlist, audience enforcement, nonce plus PKCE. Weak and default credentials rejected at every signing entrypoint. A durable, attributable audit trail. Around **80 hardening changes since March 2026**, and this work runs continuously in the background, alongside the feature releases above rather than instead of them.

Where We're Going

As of September 2026, and subject to change. Themes, not dates. Each one is tracked in the open on GitHub.

Registry Copilot

An embedded chat panel that answers "how do I" and "which asset does X" against the live catalog, then an agent builder: describe the agent you want, get the registry assets that fulfill it plus a deployable spec (YAML AgentSpec and an Amazon Bedrock AgentCore harness).

Frictionless identity for coding assistants

The registry publishes its own Client ID Metadata Document (CIMD) as of 1.31.0, shipping disabled. Next is the identity-provider half plus ID-JAG (RFC 8693 token exchange), so a coding assistant, the enterprise identity provider, and this gateway connect with the least friction on every IdP, with no per-client registration to maintain on either side.

Scale and latency

Moving the performance-intensive pieces to Go, starting with the auth validation hop that runs on every single request, with the Python path kept as the fallback. Pooled connections on the egress hot path landed in 1.31.0. Still to come: native text indexes for the keyword half of hybrid search.

Federation with SaaS agent catalogs

We federate with the Workday Agent System of Record (ASOR) today. Next: Salesforce, SAP, and other vendor catalogs, with ARD as the common mechanism, so enterprise agents discover vendor-hosted agents and tools without a bespoke integration per vendor.

Machine identities (SPIFFE)

Workload identity for agents and for the gateway's own hops: short-lived attested SVIDs issued by SPIRE or an Athenz-style issuer, instead of long-lived shared secrets that leak and never rotate.

This roadmap sits **on top of our usual biweekly releases**, which carry bug fixes, the small features customers ask for, and community contributed enhancements. 1.31.0 shipped the week of September 21, 2026; the next one is **1.32.0**.

We build in the open, tell us what you need →

Community Adoption

A community-driven project with growing momentum since June 2025

936	62	239	2,300
GitHub Stars	**Contributors**	**Forks**	**Lifetime Deployments**

GitHub Stars -- Monthly Growth



973	535	35	27.0%	ECS ≈ K8s ≈ Docker
Merged PRs	Issues Closed	Releases	Customers running 3+ days	among active customers

Listed on GoodAIList — curated directory of top AI repositories

Customer Stories

We are collecting stories from teams deploying AI Registry in production.

If your organization is using MCP Gateway Registry and would like to share your experience, we would love to hear from you.

COMING SOON

Share your story on GitHub Discussions →

Featured Coverage

AWS MACHINE LEARNING BLOG · MAY 2026

Securing AI agents: How AWS and Cisco AI Defense scale MCP and A2A deployments

A joint AWS and Cisco perspective on how the MCP Gateway & Registry, with Cisco AI Defense scanners, secures the supply chain for MCP servers and A2A agents at enterprise scale -- registration-time scanning, runtime trust, and federation across AWS Agent Registry, Anthropic, and Workday ASOR.

Read on aws.amazon.com →

AWS OPEN SOURCE BLOG · JUNE 2026

Governing AI assets at scale with MCP Gateway and Registry

How the MCP Gateway & Registry gives enterprises one solution to discover, govern, and securely connect MCP servers, A2A agents, and skills -- unified semantic discovery, OAuth2 across IdPs, visibility controls, and federation, all open source.

Read on aws.amazon.com →



06

Get Started

Try it, connect a coding assistant, and get in touch.

Demo & Links

★ **Securing AI agents: How AWS and Cisco AI Defense scale MCP and A2A deployments**

AWS Machine Learning Blog -- joint AWS + Cisco perspective

▶ **AWS Show & Tell**

YouTube -- MCP Gateway & Registry deep dive

▶ **Claude Code + DCR Demo**

Vidcast -- One-command OAuth integration walkthrough

▶ **Codex CLI + DCR Demo**

Vidcast -- OpenAI Codex connecting via DCR OAuth

▶ **Platform Demo**

Vidcast -- Hands-on walkthrough of the platform

☰ **Roadmap**

Scan to Get Started

github.com/agentic-community/mcp-gateway-registry

Open source, Apache 2.0 License

AI Gateway & Registry

One AI Registry. Unified Control.

3	6	Open
AWS Targets	**Identity Providers**	**Source**
ECS Fargate, EKS, EC2	Keycloak, Entra ID, Okta, Auth0, Cognito, PingFederate	Apache 2.0 License

github.com/agentic-community/mcp-gateway-registry

Quick Start | **Documentation** | **AWS Deployment**

AWS Workshop | **Demo Video** | **Community**